

Instant Academy Schema

INSTANT

WHERE, LIKE & BETWEEN

Data Analysis Diploma • Session 25

READING DATA WITH SELECT QUERIES

Filtering · Logical Operators · Pattern Matching · Aggregate Functions · GROUP
BY · HAVING · Executive KPIs

8 Tables

Relational Schema

Students, Courses,
Payments...

30 Slides

Interactive Presentation

Complete SQL Querying Guide

5 KPIs

Executive Dashboards

Revenue, Leads, Attendance...

[Start Session 25 →](#)[GROUP BY & HAVING Pipeline](#)[Production KPI Dashboards](#)

— OUR DATABASE SCHEMA (QUICK RECAP)

Instant Academy — Our Database Schema (Quick Recap)

All examples in this session use the Instant Academy dataset. Here are the 8 tables we work with:

All examples in this session use the [Instant Academy dataset](#). Click any table to inspect:

Students - StudentID (PK) - StudentName - Email - City - EnrollDate	Courses - CourseID (PK) - CourseName - Category - Price - DurationHrs	Enrollments - EnrollmentID (PK) - StudentID (FK) - CourseID (FK) - EnrollDate - EnrollStatus	Payments - PaymentID (PK) - EnrollmentID (FK) - Amount - PaymentDate - Method - Status
Leads - LeadID (PK) - AgentID (FK) - CampaignID (FK) - LeadName - Source - Status - CreatedDate	Campaigns - CampaignID (PK) - AgentID (FK) - CampaignName - Channel - Budget - StartDate	Attendance - AttendanceID (PK) - EnrollmentID (FK) - AttendStatus - Notes	SalesAgents - AgentID (PK) - AgentName - Email - Region - MonthlyTarget

— SYNTAX & CLAUSE ANATOMY

SELECT — Syntax & Anatomy

The most important statement in SQL for data analysts

SQL Syntax & Anatomy

```
SELECT column1, column2, ... -- Which columns to show
FROM TableName -- Which table to read from
WHERE condition -- Filter rows (optional)
ORDER BY column ASC/DESC; -- Sort results (optional)
```

SELECT

Specifies which columns appear in your result. Use ***** for all columns (avoid in production).

FROM

Names the table you're querying. The database engine reads and extracts stored records from here.

WHERE

Filters rows before returning them. Only rows matching the boolean condition are included in output.

ORDER BY

Sorts the final result set. **ASC** = ascending (default), **DESC** = descending.

— PERFORMANCE & READABILITY

SELECT * vs Specific Columns

Why being specific matters for performance, network transfer, and maintainability

⊗ SELECT * (Avoid in Production)

```
SELECT * FROM Students;
```

- **Returns ALL columns** — transfers unnecessary data
- **Slower execution** — wastes RAM and network bandwidth
- **Fragile application code** — breaks apps if order changes
- **Harder to read results** — clutters client reports
- **Security risk** — may leak private columns

☑ SELECT Specific Columns (Best Practice)

```
SELECT StudentID, StudentName, Email, City  
FROM Students;
```

- **Returns only what you need** — lean and focused
- **Faster & efficient** — minimum data transferred
- **Explicit and readable** — colleagues see required fields
- **Safer** — prevents accidental sensitive data leaks
- **Industry standard** — required in all clean SQL

💡 **Exception:** Use **SELECT *** only during quick exploratory queries in dev/test — never in production backends.

FILTERING ROWS & EXECUTION ORDER

The WHERE Clause — Filtering Rows

The WHERE clause filters rows returned by a SELECT statement. Only rows that satisfy the condition are included.

What is WHERE?

The **WHERE** clause filters rows returned by a SELECT statement. Only rows that satisfy the boolean condition are included in the result.

```
SELECT column1, column2... FROM TableName WHERE condition;
```

SQL Execution Order

- 1 FROM → (identify source table)
- 2 WHERE → (filter individual rows)
- 3 SELECT → (pick specified columns)
- 4 ORDER BY → (sort final result set)

Ex 1 — Active Students

```
SELECT StudentID, City
FROM Students
WHERE Status = 'Active';
```

Ex 2 — Price > 3,000 EGP

```
SELECT CourseName, Price
FROM Courses
WHERE Price > 3000;
```

Ex 3 — Enrolled in 2024

```
SELECT StudentName, EnrollDate
FROM Students
WHERE EnrollDate >= '2024-01-01';
```

⚠ Text requires quotes 'Active' | Numbers no quotes Price > 3000 | Dates use 'YYYY-MM-DD'

— AND · OR · NOT PRECEDENCE

Logical Operators — AND, OR, NOT

Combining multiple filtering criteria using boolean logic and enforcing correct precedence with parentheses

AND

ALL MUST BE TRUE

TRUE AND TRUE → **TRUE** ✓
TRUE AND FALSE → **FALSE** ✗

```
WHERE Status = 'Active'
AND City = 'Cairo';
```

OR

AT LEAST ONE TRUE

TRUE OR FALSE → **TRUE** ✓
FALSE OR FALSE → **FALSE** ✗

```
WHERE Source = 'Social Media'
OR Source = 'Email';
```

NOT

REVERSES CONDITION

NOT TRUE → **FALSE** ✗
NOT FALSE → **TRUE** ✓

```
WHERE NOT EnrollStatus = 'Pending';
-- Same as: WHERE EnrollStatus <> 'Pending'
```

⚠ Combining AND + OR — Always Use Parentheses!

```
SELECT StudentName, City, Status FROM Students
WHERE Status = 'Active' AND (City = 'Cairo' OR City = 'Alexandria');
```

Precedence: SQL evaluates **NOT** first, then **AND**, then **OR**. Always use parentheses **()** to guarantee intended logic.

BETWEEN & IN OPERATORS

BETWEEN and IN — Range and List Filtering

Cleaner, faster, and more readable alternatives to chained relational comparisons

BETWEEN Inclusive Range Filter

Returns rows where value falls within range (inclusive of both endpoints).

```
-- Price between 1000 and 5000 EGP
SELECT CourseName, Price FROM Courses
WHERE Price BETWEEN 1000 AND 5000;
```

```
-- Students enrolled in Q1 2024
SELECT StudentName, EnrollDate FROM Students
WHERE EnrollDate BETWEEN '2024-01-01' AND '2024-03-31';
```

Tip: BETWEEN is INCLUSIVE – includes both 1000 and 5000 endpoints.

IN List Membership Filter

Returns rows where value matches ANY item in list. Much cleaner than multiple ORs.

```
-- Students from specific cities
SELECT StudentName, City FROM Students
WHERE City IN ('Cairo', 'Alex', 'Giza');
```

```
-- Leads with specific statuses
SELECT LeadName, Status FROM Leads
WHERE Status IN ('New', 'Interested');
```

Clean SQL: WHERE City IN ('Cairo', 'Alex') replaces lengthy OR chains.

Exclusion with NOT: WHERE Price NOT BETWEEN 1000 AND 5000 | WHERE City NOT IN ('Cairo', 'Alex')

WILDCARDS % & _

LIKE — Pattern Matching with Wildcards

Search for patterns in text columns — essential for partial name searches, email domains, and SKU codes.

%

Percent Sign (%) — Multi-character Wildcard

Matches **zero or more** characters (e.g. 'A%' starts with A, '%SQL%' contains SQL)

_

Underscore () — Single-character Wildcard

Matches **exactly ONE** character (e.g. '_ohamed', 'SA____' = SA + 4 chars)

Test Pattern on Sample Dataset:

Mo%

%Data%

%@gmail.com

SA_

```
SELECT * FROM Records WHERE Value LIKE 'Mo%';
```

Mohamed Ali

m.ali@gmail.com
Data Analysis Track
SA1042 ✓ Match

Mona Youssef

mona@yahoo.com
Advanced SQL Masterclass
SA9821 ✓ Match

Ahmed Hassan

ahmed@gmail.com
Python for Data Science
PY2031

Sara Kamel

sara@academy.org
Excel Analytics Pro
EX4011

⚠ **Performance Note:** Leading wildcards like '%SQL' cannot use B-Tree indexes and scan entire tables. Prefix searches like 'SQL%' utilize index seeks.

HANDLING NULL VALUES & MISSING DATA

NULL Values — Understanding Missing Data

NULL means the value is UNKNOWN or MISSING. It is NOT zero, NOT empty string, NOT false.

? What is NULL?

NULL means **UNKNOWN** or **MISSING**. It is NOT zero (0), NOT empty string (""), and NOT false.

- Student with no NationalID entered yet
- Lead with no marketing campaign assigned
- Attendance record with no notes recorded

⚠ The NULL Trap

You **CANNOT** compare NULL with = or <>. These always return UNKNOWN (neither TRUE nor FALSE):

```
X WHERE NationalID = NULL -- returns 0 rows always!
✓ WHERE NationalID IS NULL -- CORRECT
```

IS NULL & IS NOT NULL:

```
-- Find students with no NationalID
SELECT StudentName, NationalID FROM Students
WHERE NationalID IS NULL;

-- Find leads assigned to a campaign
SELECT LeadName, CampaignID FROM Leads
WHERE CampaignID IS NOT NULL;
```

ISNULL() & COALESCE():

```
-- Replace NULL with default value
SELECT StudentName,
       ISNULL(NationalID, 'Not Provided') AS NationalID
FROM Students;

-- COALESCE: returns first non-NULL value
SELECT LeadName, COALESCE(CampaignID, AgentID, 0) AS AssignedTo FROM
Leads;
```

Calculations: NULL + 10 = NULL | SUM() ignores NULLs | COUNT(*) counts all rows, COUNT(col) skips NULLs

— ORDER BY ASC / DESC

ORDER BY — Sorting Query Results

Sorts the rows returned by your query. Can sort by one or more columns, ascending (ASC) or descending (DESC).
Applied LAST.

Sort By:

PRICE

NAME

DATE

Direction:

ASC (A→Z, 1→100)

DESC (Z→A, 100→1)

```
SELECT CourseName, Price, EnrollDate FROM Courses ORDER BY price DESC;
```

CourseName	Price (EGP)	EnrollDate
Power BI Master	5,000	2024-02-10
SQL Analysis	4,000	2024-01-15
Python Basics	2,500	2024-03-01
Excel Formulas	1,500	2024-04-05

 **Pro Tip:** You can ORDER BY a column not listed in SELECT. SQL sorts by it internally without displaying it.

TOP N & DISTINCT DEDUPLICATION

TOP and DISTINCT — Limiting and Deduplicating Results

Restrict total rows returned and eliminate duplicate values from results

TOP N Return First N Rows

Use TOP to limit results to the first N rows (always pair with ORDER BY for deterministic results).

```
-- Top 5 most expensive courses
SELECT TOP 5 CourseName, Price FROM Courses
ORDER BY Price DESC;
```

```
-- Top 10% highest payments
SELECT TOP 10 PERCENT Amount, Method FROM Payments
ORDER BY Amount DESC;
```

Remember: TOP without ORDER BY yields arbitrary non-reproducible rows!

DISTINCT Remove Duplicate Rows

Removes duplicate rows from query results, showing each unique value or combination only once.

```
-- What cities do students come from?
SELECT DISTINCT City FROM Students
ORDER BY City;
```

```
-- Distinct payment methods used
SELECT DISTINCT Method, Status FROM Payments;
```

Combine Both: SELECT DISTINCT TOP 5 City FROM Students WHERE Status = 'Active' ORDER BY City;

— PUTTING IT ALL TOGETHER

Putting It All Together — Building Complex Queries

Combining WHERE, AND/OR, BETWEEN, IN, LIKE, IS NULL, and ORDER BY in single powerful queries:

Business Q1

\$ Business Q2

⚠ Business Q3

Business Q1: Active Cairo Students in 2024

Ready to execute

```
SELECT StudentName, City, EnrollDate, Status
FROM Students
WHERE Status = 'Active'
      AND City = 'Cairo'
      AND EnrollDate BETWEEN '2024-01-01' AND '2024-12-31'
ORDER BY StudentName ASC;
```

Combines: WHERE, AND/OR, ORDER BY

QUERY OUTPUT PREVIEW (TOP MATCHING ROWS)

StudentName	City	EnrollDate	Status
Ahmed Hassan	Cairo	2024-02-14	Active
Karim Fouad	Cairo	2024-05-20	Active
Sara Mahmoud	Cairo	2024-08-11	Active

Execution Note: Filter is executed on database server disk, returning only lean data across the network connection.

COMMON MISTAKES & PRO TIPS

Common Mistakes & Pro Tips

5 classic pitfalls to avoid when writing filtering and sorting queries in SQL Server

✗ `WHERE NationalID = NULL`

✓ `WHERE NationalID IS NULL`

! = NULL always returns 0 rows (three-valued logic)

✗ `WHERE Price = '3000'`

✓ `WHERE Price = 3000`

! Numbers do NOT use quotes; causes implicit conversion

✗ `WHERE Status = active`

✓ `WHERE Status = 'Active'`

! Strings must be wrapped in single quotes

✗ `WHERE City = 'Cairo' OR City = 'Giza' OR City = 'Alex'`

✓ `WHERE City IN ('Cairo', 'Giza', 'Alex')`

! IN operator is cleaner, less prone to logic errors, and faster

✗ `ORDER BY` without `WHERE` on large table

✓ Add `WHERE` to filter rows before sorting

! Sorting millions of unfiltered rows consumes immense memory

FROM RAW ROWS TO BUSINESS INSIGHT

Why Aggregate? — From Raw Rows to Business Insight

Raw data is an INPUT — aggregation turns thousands of unorganized rows into actionable BUSINESS ANSWERS.

⊗ RAW DATA — NOT USEFUL FOR DECISION MAKERS

EnrollID	StudentID	Amount	Status
1001	S01	2500	Paid
1002	S02	3200	Paid
1003	S01	1800	Pending
1004	S03	2500	Paid
1005	S04	4000	Paid

Thousands of raw rows. No immediate totals, no summaries, no insight. You cannot answer "What is our revenue?" by scrolling through rows.

Core Rule: Raw data is an *INPUT* — aggregation turns it into a *BUSINESS ANSWER*.

⊙ AGGREGATED ANSWERS — IMMEDIATE EXECUTIVE CLARITY

Total Revenue? EGP 1,234,500

Average Payment? EGP 3,200

Total Enrollments? 1,842

Attendance Rate? 87.3%

Leads Converted? 34.2%

THE 5 CORE AGGREGATE FUNCTIONS

The 5 Core Aggregate Functions

COUNT(), SUM(), AVG(), MIN(), MAX() — the mathematical building blocks of every SQL analyst dashboard

COUNT()

Counts rows or non-NULL values

COUNT(*) → all rows
COUNT(City) → non-null
COUNT(DISTINCT City)

SUM()

Total sum of numeric column

SUM(Amount) → total revenue
SUM(Price) → total catalog
SUM(Budget) → marketing

AVG()

Arithmetic average of column

AVG(Amount) → avg payment
AVG(Price) → avg price
AVG(MonthlyTarget)

MIN()

Smallest value in column

MIN(Price) → cheapest
MIN(EnrollDate) → 1st date
MIN(Amount) → min paid

MAX()

Largest value in column

MAX(Amount) → top payment
MAX(Price) → flagship
MAX(HireDate) → newest

⚠ Critical Warning: Aggregate functions **CANNOT** appear in a **WHERE** clause — because WHERE filters row-by-row before aggregates are computed. Use **HAVING** instead!

COUNTING ACADEMY RECORDS

COUNT() — Counting Academy Records

Understand the critical difference between COUNT(*), COUNT(col), and COUNT(DISTINCT col)

THREE FLAVORS OF COUNT

Syntax	What it Counts
<code>COUNT(*)</code>	All rows (including NULLs)
<code>COUNT(col)</code>	Non-NULL values only (skips NULLs)
<code>COUNT(DISTINCT col)</code>	Unique non-NULL values only

Academy Questions Answered:

- How many students are in database? → `COUNT(*)`
- How many active leads does team manage? → `COUNT(LeadID) WHERE`
- How many distinct cities are students from? → `COUNT(DISTINCT City)`

Rule: `COUNT(col)` skips NULLs — `COUNT(*)` does not. Use `COUNT(*)` for total row count.

```
-- Total students in database
SELECT COUNT(*) AS TotalStudents FROM Students;

-- Active students only
SELECT COUNT(*) AS ActiveStudents
FROM Students WHERE Status = 'Active';

-- Distinct cities where students live
SELECT COUNT(DISTINCT City) AS UniqueCities FROM Students;

-- Count total leads vs leads with source
SELECT COUNT(LeadID) AS TotalLeads,
       COUNT(Source) AS WithSource FROM Leads;
```


— REVENUE AND PRICING ANALYSIS

SUM() & AVG() — Revenue and Pricing Analysis

Calculating financial totals, averages, minimums, maximums, and transaction counts across tables

```
-- Full payment summary from Payments table
SELECT
  SUM(Amount) AS TotalRevenue,
  AVG(Amount) AS AvgPayment,
  MIN(Amount) AS MinPayment,
  MAX(Amount) AS MaxPayment,
  COUNT(*) AS Transactions
FROM Payments
WHERE Status = 'Paid';

-- Average course price per category
SELECT Category, AVG(Price) AS AvgPrice
FROM Courses GROUP BY Category;
```

SAMPLE OUTPUT — PAYMENTS TABLE METRICS

EGP 1.23M

Total Revenue

EGP 3,200

Avg Payment

EGP 500

Min Payment

EGP 9,800

Max Payment

1,842

Total Paid Transactions

Rule: NULL values are automatically excluded from SUM() and AVG().

— THE BACKBONE OF BUSINESS REPORTING

GROUP BY — The Backbone of Business Reporting

Bucketing individual rows into distinct groups so aggregate functions compute one summary result per category

All Rows
(Enrollments)

GROUP BY
CourseID

Aggregate
per Group

One Row
per Course

```
-- Enrollments per course
SELECT CourseID, COUNT(*) AS Enrollments
FROM Enrollments
GROUP BY CourseID;

-- Total revenue per payment method
SELECT Method,
       COUNT(*) AS Transactions,
       SUM(Amount) AS Revenue,
       AVG(Amount) AS AvgPayment
FROM Payments
GROUP BY Method
ORDER BY Revenue DESC;
```

GROUP BY GOLDEN RULES

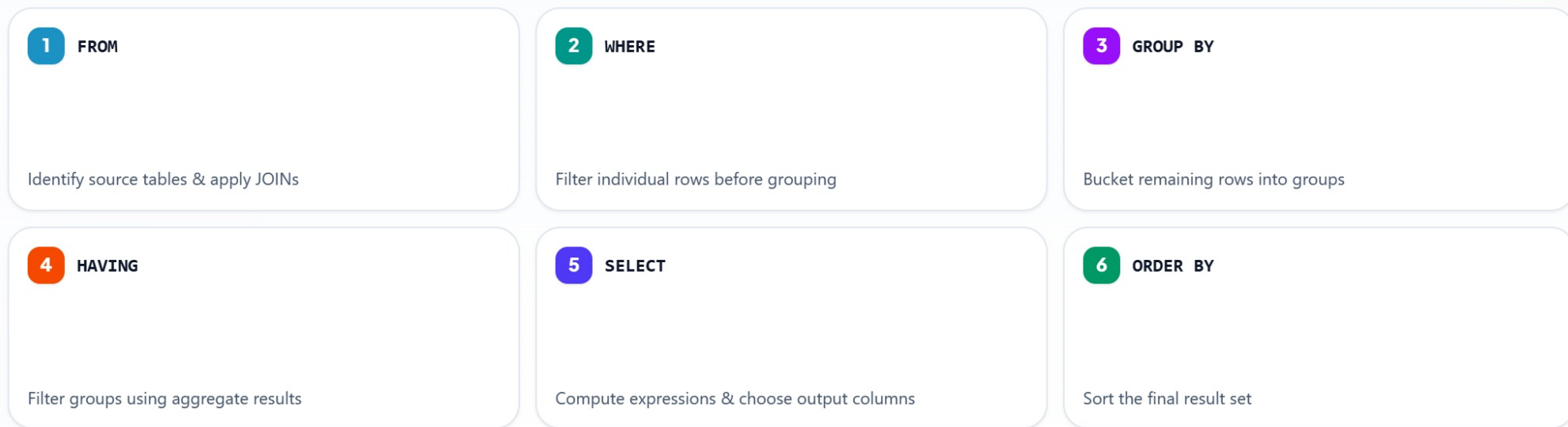
- 1 Every **non-aggregated column** in SELECT must appear in GROUP BY.
- 2 You **CAN** group by multiple columns simultaneously.
- 3 GROUP BY runs **AFTER WHERE** (filters rows first).
- 4 Output has **exactly ONE row** per unique group value.
- 5 Combine with **ORDER BY** to rank summary metrics.

SQL EXECUTION ORDER (CRITICAL CONCEPT)

SQL Execution Order — What Runs When

Why WHERE cannot reference aliases or aggregates: the database engine executes clauses in a strict non-written order

SQL is non-procedural: you write queries in one order, but the database engine executes them in a strict chronological sequence:



Why This Matters:

WHERE cannot reference aggregates because it runs at Step 2 (before GROUP BY at Step 3). HAVING runs at Step 4, specifically to filter after aggregates exist!

COMBINING WHERE AND GROUP BY

Combining WHERE and GROUP BY

Filter individual rows first with WHERE, then bucket remaining rows with GROUP BY, and finally aggregate

```
-- Total payments per method (filtered then grouped)
SELECT Method,
       COUNT(PaymentID) AS TotalPayments,
       SUM(Amount) AS TotalRevenue,
       AVG(Amount) AS AvgPayment
FROM Payments
WHERE Status = 'Paid'
GROUP BY Method
ORDER BY TotalRevenue DESC;
```

Execution: 1. Excludes unpaid rows → 2. Groups remaining by Method → 3. Calculates totals → 4. Sorts output.

WHERE VS GROUP BY VS HAVING

Clause	Filters	Timing
WHERE	Individual rows	BEFORE grouping
GROUP BY	Creates groups	After WHERE
HAVING	Grouped results	AFTER aggregation

Quick Decision Drills:

- Filter students enrolled after 2024? → `WHERE EnrollDate > '2024-01-01'`
- Only courses with 50+ enrollments? → `HAVING COUNT(*) >= 50`
- Exclude cancelled payments first? → `WHERE Status <> 'Cancelled'`

FILTERING AGGREGATED GROUPS

HAVING — Filtering Aggregated Groups

HAVING filters GROUPS after aggregation. It can reference aggregate results — WHERE cannot.

```
-- Enrollments grouped by status with 30+ students
SELECT EnrollStatus, COUNT(EnrollmentID) AS Total
FROM Enrollments
GROUP BY EnrollStatus
HAVING COUNT(EnrollmentID) >= 30
ORDER BY Total DESC;

-- Leads grouped by channel having > 50 leads
SELECT Source, Status, COUNT(LeadID) AS TotalLeads
FROM Leads
GROUP BY Source, Status
HAVING COUNT(LeadID) > 50;
```

COMMON HAVING PATTERNS

HAVING COUNT(*) > 10	Groups with more than 10 rows
HAVING SUM(Amount) > 100000	Groups with high total revenue
HAVING AVG(Price) < 5000	Affordable course categories
HAVING COUNT(DISTINCT City) >= 3	Courses reaching 3+ cities
HAVING MIN(Amount) >= 1000	All payments in group at least 1,000
HAVING MAX(Budget) < 50000	Low-spend marketing campaigns only

— WHERE VS HAVING COMPARISON

WHERE vs HAVING — Know the Difference

A side-by-side breakdown of the most tested SQL concept in technical interviews

WHERE Clause

Filters: ROWS before grouping

Aggregates: **CANNOT** use aggregate functions

Timing: Runs BEFORE GROUP BY

Scope: Works on raw column values

Example: `WHERE Status = 'Active'`

HAVING Clause

Filters: GROUPS after aggregation

Aggregates: **CAN** and **MUST** use aggregate functions

Timing: Runs AFTER GROUP BY

Scope: Works on computed group metrics

Example: `HAVING COUNT(*) > 50`

Synthesis: Both can coexist: `WHERE` filters rows first → `GROUP BY` groups → `HAVING` filters groups.

TOTAL & MONTHLY REVENUE (PAYMENTS TABLE)

KPI 1 — Total & Monthly Revenue

Executive revenue summary and chronological monthly financial trend breakdown

```
-- Monthly revenue breakdown query
SELECT
  YEAR(PaymentDate) AS Yr,
  MONTH(PaymentDate) AS Mo,
  SUM(Amount) AS Revenue,
  COUNT(*) AS Transactions
FROM Payments
WHERE Status = 'Paid'
GROUP BY YEAR(PaymentDate), MONTH(PaymentDate)
ORDER BY Yr, Mo;
```

REVENUE DASHBOARD

EGP 1.23M

Total Revenue (YTD)

EGP 102K

Avg Monthly Revenue

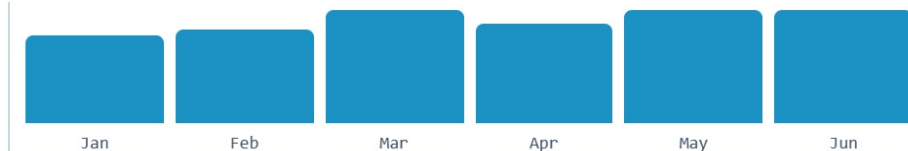
1,842

Total Transactions

EGP 3,200

Avg Transaction

Monthly Revenue (EGP)



— ENROLLMENTS PER COURSE & CATEGORY

KPI 2 — Enrollments per Course

Measuring academic demand, enrollment volume, and distribution by curriculum category

```
-- Enrollments grouped by status
SELECT
  EnrollStatus,
  COUNT(EnrollmentID) AS TotalEnrollments,
  MIN(EnrollDate) AS FirstEnroll,
  MAX(EnrollDate) AS LastEnroll,
  COUNT(DISTINCT CourseID) AS CoursesActive
FROM Enrollments
GROUP BY EnrollStatus
HAVING COUNT(EnrollmentID) > 0
ORDER BY TotalEnrollments DESC;
```

SAMPLE OUTPUT — COURSE ENROLLMENTS

Course	Category	Enrolled	Revenue
SQL Analysis	Programming	412	EGP 824K
Power BI	Data Science	356	EGP 712K
Excel Analytics	Business	298	EGP 447K
Python Data	Programming	241	EGP 482K

Enrollments by Category

Data Science 33%

Programming 29%

Business 24%

Other 14%

— ATTENDANCE COUNT PER COURSE & STATUS

KPI 3 — Attendance Count per Course

Tracking student retention, present vs absent counts, and engagement across active cohorts

```
-- Attendance summary by status
SELECT AttendStatus,
       COUNT(AttendanceID) AS TotalRecords,
       COUNT(DISTINCT EnrollmentID) AS UniqueEnrollments
FROM Attendance
GROUP BY AttendStatus
ORDER BY TotalRecords DESC;

-- Present-only attendance count
SELECT AttendStatus, COUNT(AttendanceID) AS PresentCount
FROM Attendance WHERE AttendStatus = 'Present' GROUP BY AttendStatus;
```

ATTENDANCE DASHBOARD

9,842

Total Sessions Logged

8,591

Present Count (87.3%)

1,251

Absent Count

41

Courses Tracked

Present Count by Track



— LEAD FUNNEL BY STATUS & CAMPAIGN

KPI 4 — Lead Count by Status & Campaign

Marketing conversion funnel analysis from initial inquiry to enrolled student

```
-- Leads per source with date range
SELECT
  Source, Status,
  COUNT(LeadID) AS TotalLeads,
  MIN(CreatedDate) AS FirstLead,
  MAX(CreatedDate) AS LastLead
FROM Leads
WHERE Status <> 'Cancelled'
GROUP BY Source, Status
HAVING COUNT(LeadID) >= 10
ORDER BY TotalLeads DESC;
```

LEAD COUNT BY FUNNEL STATUS



Key Insight: `GROUP BY Status` reveals the funnel distribution.
`MIN/MAX(CreatedDate)` tracks campaign velocity.

SALES AGENT SCORECARD & TARGET ATTAINMENT

KPI 5 — Sales Agent Performance

Individual and regional revenue generation measured against monthly assigned targets

```
-- Active agents grouped by region
SELECT
  Region,
  COUNT(AgentID) AS TotalAgents,
  AVG(MonthlyTarget) AS AvgTarget,
  SUM(MonthlyTarget) AS TotalTarget,
  MAX(MonthlyTarget) AS TopAgent
FROM SalesAgents
WHERE IsActive = 1
GROUP BY Region
ORDER BY TotalTarget DESC;
```

AGENT SCORECARD

Agent	Leads	Revenue	Target
Hana R.	91	EGP 190K	127%
Ahmed K.	87	EGP 180K	120%
Sara M.	72	EGP 140K	93%
Karim F.	65	EGP 105K	70%

Agent Revenue Attainment

Hana 190K

Ahmed 180K

Sara 140K

Karim 105K

FULL KPI DASHBOARD — EXECUTIVE SUMMARY

Building a Full KPI Dashboard — Executive Summary

One master script replaces 4 separate manual reports — this is what modern management dashboards are built from.

```
-- KPI 1: Student summary
SELECT COUNT(StudentID) AS TotalStudents,
       COUNT(DISTINCT City) AS Cities FROM Students;

-- KPI 2: Enrollment summary
SELECT COUNT(EnrollmentID) AS TotalEnrollments,
       COUNT(DISTINCT StudentID) AS UniqueStudents FROM Enrollments;

-- KPI 3: Revenue summary
SELECT COUNT(PaymentID) AS Transactions,
       SUM(Amount) AS TotalRevenue FROM Payments
WHERE Status = 'Paid';
```

EXECUTIVE KPI RESULTS

2,847

Total Students

1,842

Enrollments

EGP 1.23M

Total Revenue

9,842

Sessions Logged

4,120

Total Leads Generated

One master script replaces 4 separate reports. This is what modern executive dashboards are built from!

— AVOID THESE SQL ERRORS

Common Mistakes — Avoid These SQL Errors

Fixing the two most frequent aggregation errors: missing GROUP BY columns and placing aggregates in WHERE.

MISTAKE 1: MISSING COLUMN IN GROUP BY ('COLUMN IS INVALID IN SELECT LIST')

✗ WRONG — Column 'Category' missing:

```
SELECT CourseName, Category, COUNT(*)
FROM Courses
GROUP BY CourseName;
```

✓ CORRECT — All columns included:

```
SELECT CourseName, Category, COUNT(*)
FROM Courses
GROUP BY CourseName, Category;
```

MISTAKE 2: AGGREGATE FUNCTION IN WHERE CLAUSE ('AN AGGREGATE MAY NOT APPEAR IN WHERE')

✗ WRONG — COUNT(*) inside WHERE:

```
SELECT CourseID, COUNT(*)
FROM Enrollments
WHERE COUNT(*) > 10
GROUP BY CourseID;
```

✓ CORRECT — Moved to HAVING:

```
SELECT CourseID, COUNT(*)
FROM Enrollments
GROUP BY CourseID
HAVING COUNT(*) > 10;
```

💡 **Quick Fix:** When you see 'invalid in select list' — check that every non-aggregate SELECT column appears in GROUP BY.

INSTANT

✓ SESSION 25 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

🎉 **Session 25 Complete! You have mastered SQL SELECT Queries, Filtering, Aggregations, GROUP BY, HAVING & Business KPIs!**

↺ Restart Presentation

Open Session 26: SQL Built-in Functions & CASE WHEN →